



Departamento de Programación
Facultad de Informática
Universidad Nacional del Comahue



Programación Concurrente

LEER APUNTE PARTE V DE PEDCO



Sincronización competencia - Cerrojos

Sincronización cooperación - Cerrojos + variables de condición

Exclusión mútua

Mecanismos

```
graph TD; A[Mecanismos] --> B[Semáforos]; A --> C[Métodos y Bloques sincronizados]; A --> D[Cerrojos];
```

Semáforos

Métodos y Bloques sincronizados

Cerrojos

Mecanismo del cerrojo

- Mecanismo simple basado en cerrojos

Adquirir cerrojo

SECCION CRITICA

Liberar cerrojo

SECCION RESTANTE

- Las operaciones de adquisición y liberación del cerrojo han de ser atómicas (su ejecución ha de producirse en una unidad de trabajo indivisible).

Cerrojo – Interfaz Lock de Java

- Mecanismo de sincronización de hilos, es más flexible y sofisticado que los métodos sincronizados, pero menos eficiente.
- La interfaz Lock define un conjunto de operaciones abstractas de toma y liberación de un lock.
- las operaciones de suspensión y liberación de un lock **son explícitas**.



A Investigar!!

Interfaz Lock

Lock es una interfaz de Java dentro del package `java.util.concurrent.locks` con los siguientes métodos:

- El método *lock()*
- El método *tryLock()*
- El método *unlock()*

ReentrantLock es una implementación de la interfaz Lock de Java dentro del package **`java.util.concurrent.locks`**

Comparamos...

sincronización por cooperación

- **Bloques sincronizados**
- **Métodos sincronizados**
- **Locks explícitos/cerrojos**
- **Semáforos**



Sincronización competencia y cooperación

Mecanismos

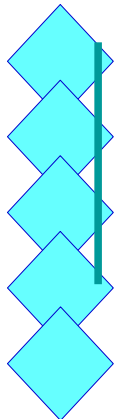
```
graph TD; A[Mecanismos] --> B[Monitores y esperas guardadas]; A --> C[Semáforos binarios y generales]; A --> D[Cerrojos + Variables de condición]; A --> E[Otras posibilidades ...];
```

Monitores y esperas guardadas

Semáforos binarios y generales

Cerrojos + Variables de condición

Otras posibilidades ...



Interfaz Condition

También conocido como
colas de condición o variables de condición

- Hacen que un hilo suspenda la ejecución hasta que otro hilo avise alguna condición de estado como cierta.
- El acceso al objeto protegido se produce en diferentes lugares
- El bloqueo está asociado a la Condición
- El hilo que debe esperar por una condición libera atómicamente el bloqueo asociado y se suspende

Variables de condición

asociadas a un reentrantLock

Operaciones

(throws InterruptedException)

- void ***await()***
- boolean ***await(long time, TimeUnit unit)***

- void ***signal()*** *//Despierta un hilo*
- void ***signalAll()*** *//Despierta todos los hilos*

Problema Prod/Cons

objetos *Condition*

Cuando se utiliza un Lock explícito para definir una **región crítica**, dentro de ella se utilizan los objetos **Condition** como mecanismo de sincronización de cooperación entre hilos.

```
class BufferDatos {  
    Lock lock = new ReentrantLock(true);  
    Condition noLleno = lock.newCondition();  
    Condition noVacio = lock.newCondition();
```

- Un objeto Condition está estructuralmente ligado a un objeto Lock
 - Se crea mediante ***newCondition()*** sobre un objeto Lock.
- El objeto Condition solo puede ser invocado por un hilo que previamente haya tomado el Lock al que pertenece.

Problema Prod/Cons objetos *Condition*

```
class BufferDatos {  
    Lock lock = new ReentrantLock(true);  
    Condition noLleno = lock.newCondition();  
    Condition noVacio = lock.newCondition();
```

Los hilos productores quedarán bloqueados en espera de lugar en la cola de espera de la variable de condición “noLleno” , *noLleno.await()*

Los hilos consumidores quedarán bloqueados en espera de items para consumir en la cola de espera de la variable de condición “noVacio”, *noVacio.await()*

noLleno.signal()
noLleno.signalAll()

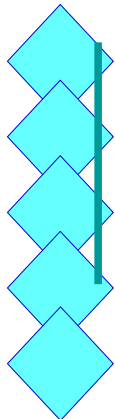


```
import java.util.concurrent.locks.*;

class buffer_limitado {
    final Lock cerrojo = new ReentrantLock();
    final Condition noLlena = cerrojo.newCondition();
    final Condition noVacía = cerrojo.newCondition();
    final Object[] items = new Object[100];

    public void poner (Object x) {
        cerrojo.lock();

        ....
    }
```



```
import java.util.concurrent.locks.*;
```

```
class buffer_limitado {
```

```
    final Lock cerrojo = new ReentrantLock(); //declara y crea un cerrojo
```

```
// variables de condicion asociadas a “cerrojo” para control de buffer lleno y  
vacio
```

```
    final Condition noLleno = cerrojo.newCondition();
```

```
    final Condition noVacio = cerrojo.newCondition();
```

```
//estructura elegida para mantener los items de datos
```

```
final Object[] items = new Object[100];
```

```
... //otras variables necesarias
```

```
public void poner(Object x) {
```

```
    cerrojo.lock();
```

```
    ...
```

```
    ...
```

```
import java.util.concurrent.locks.*;
```

```
class buffer_limitado {
```

```
    final Lock cerrojo = new ReentrantLock();
```

```
    final Condition noLlena = cerrojo.
```

```
    final Condition noVacia = cerrojo.
```

```
    final Object[] items = new Object
```

```
    public void poner (Object x) thr
```

```
        cerrojo.lock();
```

```
        ....
```

```
        cerrojo.lock();
```

```
        //duerme al hilo productor si buffer lleno  
        hasta
```

```
        //ser notificado
```

```
        try {while (lleno(items))
```

```
            noLlena.await();
```

```
        }
```

```
        // se agrega el item x a la estructura
```

```
        //se despierta a un hilo consumidor  
        esperando
```

```
            noVacia.signal();
```

```
        } finally {cerrojo.unlock();}
```

```
    }
```

Clase ReentrantLock (implementa interfaz Lock)

- Librerías que utiliza
 - `import java.util.concurrent.locks.Condition;`
 - `import java.util.concurrent.locks.Lock;`
 - `import java.util.concurrent.locks.ReentrantLock;`
- Creación de objetos ReentrantLock
 - `l = new ReentrantLock(true);` //con política equitativa
 - `l = new ReentrantLock(false);` //con política aleatoria

Comparamos...

sincronización por cooperación

- **Monitores (métodos sincronizados + espera y notificación)**
- **Locks explícitos + variables de condición**
- **Semáforos binarios y generales**

